

DSA ARRAYS & STRINGS

50-Problem Interview Handbook • Levels 1–7 • Optimal C++ Solutions

Code formatting update

Every C++ source line now starts from the same leftmost code column. Only the intentional indentation inside blocks is preserved, so the code reads like a normal C++ editor instead of wandering around the page.

Level	Problems
LEVEL 1	10
LEVEL 2	10
LEVEL 3	8
LEVEL 4	6
LEVEL 5	6
LEVEL 6	5
LEVEL 7	5
TOTAL	50

LEVEL 1 • Basic Array Manipulation

1. Find Maximum Element **Absent**

Input: [3, 7, 2, 9, 4]

Output: 9

Main thing to learn: Single-pass comparison. Keep the largest value seen so far.

Pattern / technique: Linear scan

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: Assumes the array is non-empty.

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

int findMax(const vector<int>& a) {
    int mx = a[0];

    for (int x : a)
        mx = max(mx, x);

    return mx;
}

int main() {
    vector<int> a = {3, 7, 2, 9, 4};
    cout << findMax(a);
}
```

2. Find Minimum Element **Absent**

Input: [3, 7, 2, 9, 4]

Output: 2

Main thing to learn: Single-pass comparison. Keep the smallest value seen so far.

Pattern / technique: Linear scan

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: Assumes the array is non-empty.

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

int findMin(const vector<int>& a) {
    int mn = a[0];

    for (int x : a)
        mn = min(mn, x);

    return mn;
}

int main() {
    vector<int> a = {3, 7, 2, 9, 4};
    cout << findMin(a);
}
```

3. Reverse an Array **Absent**

Input: [1, 2, 3, 4, 5]

Output: [5, 4, 3, 2, 1]

Main thing to learn: Swap symmetric elements with two pointers.

Pattern / technique: Two pointers

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: In-place reversal.

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

void reverseArray(vector<int>& a) {
    int left = 0, right = a.size() - 1;

    while (left < right) {
        swap(a[left], a[right]);
        left++;
        right--;
    }
}

int main() {
    vector<int> a = {1, 2, 3, 4, 5};
    reverseArray(a);

    for (int x : a)
        cout << x << " ";
}
```

4. Check if Array is Sorted **Absent**

Input: [1, 2, 2, 4, 7]

Output: true

Main thing to learn: For ascending order, every element must be $\leq$ the next element.

Pattern / technique: Adjacent comparison

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

bool isSorted(const vector<int>& a) {
    for (int i = 1; i < a.size(); i++) {
        if (a[i] < a[i - 1])
            return false;
    }
    return true;
}

int main() {
    vector<int> a = {1, 2, 2, 4, 7};
    cout << boolalpha << isSorted(a);
}
```

5. Remove Duplicates from Sorted Array **LeetCode 26**

Input: [1,1,2,2,3]

Output: Length = 3, array begins [1,2,3]

Main thing to learn: Because the array is sorted, use a slow pointer for the next unique position.

Pattern / technique: Two pointers / slow-fast

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: LeetCode expects the unique elements in `nums[0..k-1]`.

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;
```

```

int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;

    int k = 1;

    for (int i = 1; i < nums.size(); i++) {
        if (nums[i] != nums[i - 1])
            nums[k++] = nums[i];
    }

    return k;
}

int main() {
    vector<int> nums = {1,1,2,2,3};
    int k = removeDuplicates(nums);

    cout << "Length = " << k << "\n";
    for (int i = 0; i < k; i++)
        cout << nums[i] << " ";
}

```

6. Move Zeroes [LeetCode 283](#)

Input: [0,1,0,3,12]

Output: [1,3,12,0,0]

Main thing to learn: Move every non-zero forward; fill the remaining positions with zero.

Pattern / technique: Two pointers / compaction

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: Preserves the relative order of non-zero elements.

Optimal C++

```

#include <iostream>
#include <vector>
using namespace std;

void moveZeroes(vector<int>& nums) {
    int k = 0;

    for (int x : nums) {
        if (x != 0)
            nums[k++] = x;
    }

    while (k < nums.size())
        nums[k++] = 0;
}

int main() {
    vector<int> nums = {0,1,0,3,12};
    moveZeroes(nums);

    for (int x : nums)
        cout << x << " ";
}

```

7. Rotate Array [LeetCode 189](#)

Input: [1,2,3,4,5,6,7], k = 3

Output: [5,6,7,1,2,3,4]

Main thing to learn: Reverse the whole array, then reverse the first k elements and the remaining elements.

Pattern / technique: Reversal algorithm

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: Guard $n == 0$ in production code before $k \% n$.

Optimal C++

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void rotateArray(vector<int>& nums, int k) {
    int n = nums.size();
    k %= n;

    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin() + k);
    reverse(nums.begin() + k, nums.end());
}

int main() {
    vector<int> nums = {1,2,3,4,5,6,7};
    rotateArray(nums, 3);

    for (int x : nums)
        cout << x << " ";
}

```

8. Second Largest Element Absent

Input: [10, 5, 8, 10, 3]

Output: 8

Main thing to learn: Track the largest and second-largest distinct values in one pass.

Pattern / technique: Two-value tracking

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: Assumes at least two distinct values. No exact LeetCode problem with this exact wording.

Optimal C++

```

#include <iostream>
#include <vector>
#include <climits>
using namespace std;

int secondLargest(const vector<int>& a) {
    long long largest = LLONG_MIN;
    long long second = LLONG_MIN;

    for (int x : a) {
        if (x > largest) {
            second = largest;
            largest = x;
        } else if (x > second && x < largest) {
            second = x;
        }
    }

    return (int)second;
}

int main() {
    vector<int> a = {10,5,8,10,3};
    cout << secondLargest(a);
}

```

9. Find Missing Number LeetCode 268

Input: [3,0,1]

Output: 2

Main thing to learn: The complete set is $0..n$. XOR or sum can recover the missing value.

Pattern / technique: XOR cancellation

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: XOR avoids arithmetic overflow from the sum formula.

Optimal C++

```

#include <iostream>
#include <vector>
using namespace std;

int missingNumber(vector<int>& nums) {
    int n = nums.size();
    int ans = n;

    for (int i = 0; i < n; i++)
        ans ^= i ^ nums[i];

    return ans;
}

int main() {
    vector<int> nums = {3,0,1};
    cout << missingNumber(nums);
}

```

10. Find Duplicate Number LeetCode 287

Input: [1,3,4,2,2]

Output: 2

Main thing to learn: Treat nums[value] as the next pointer. The duplicate creates a cycle; Floyd finds its entrance.

Pattern / technique: Floyd cycle detection

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: Does not modify nums. Phase 1 finds a meeting point; Phase 2 finds the cycle entrance.

Optimal C++

```

#include <iostream>
#include <vector>
using namespace std;

int findDuplicate(vector<int>& nums) {
    int slow = nums[0];
    int fast = nums[0];

    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    slow = nums[0];

    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}

int main() {
    vector<int> nums = {1,3,4,2,2};
    cout << findDuplicate(nums);
}

```

LEVEL 2 • Basic String Problems

1. Reverse String LeetCode 344

Input: "hello"

Output: "olleh"

Main thing to learn: Swap characters from both ends toward the center.

Pattern / technique: Two pointers

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```
#include <iostream>
#include <string>
using namespace std;

void reverseString(string& s) {
    int left = 0, right = s.size() - 1;

    while (left < right) {
        swap(s[left], s[right]);
        left++;
        right--;
    }
}

int main() {
    string s = "hello";
    reverseString(s);
    cout << s;
}
```

2. Reverse Words in a String **LeetCode 151**

Input: "the sky is blue"

Output: "blue is sky the"

Main thing to learn: Parse words while ignoring extra spaces, then output them in reverse order.

Pattern / technique: Tokenization + reverse order

Complexity: Time $O(n)$ • Extra space $O(n)$

Note: stringstream automatically handles repeated spaces.

Optimal C++

```
#include <iostream>
#include <sstream>
#include <vector>
using namespace std;

string reverseWords(string s) {
    stringstream ss(s);
    vector<string> words;
    string word;

    while (ss >> word)
        words.push_back(word);

    string ans;

    for (int i = words.size() - 1; i >= 0; i--) {
        if (!ans.empty()) ans += " ";
        ans += words[i];
    }

    return ans;
}

int main() {
    cout << reverseWords("the sky is blue");
}
```

3. Check Palindrome **Absent**

Input: "madam"

Output: true

Main thing to learn: Compare symmetric characters from the two ends.

Pattern / technique: Two pointers

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: Generic palindrome check. LeetCode has more specific palindrome variants.

Optimal C++

```

#include <iostream>
#include <string>
using namespace std;

bool isPalindrome(const string& s) {
    int l = 0, r = s.size() - 1;

    while (l < r) {
        if (s[l] != s[r])
            return false;
        l++;
        r--;
    }

    return true;
}

int main() {
    cout << boolalpha << isPalindrome("madam");
}

```

4. Valid Palindrome LeetCode 125

Input: "A man, a plan, a canal: Panama"

Output: true

Main thing to learn: Ignore non-alphanumeric characters and compare case-insensitively.

Pattern / technique: Two pointers + character filtering

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```

#include <iostream>
#include <string>
#include <cctype>
using namespace std;

bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;

    while (l < r) {
        while (l < r && !isalnum(s[l])) l++;
        while (l < r && !isalnum(s[r])) r--;

        if (tolower(s[l]) != tolower(s[r]))
            return false;

        l++;
        r--;
    }

    return true;
}

int main() {
    cout << boolalpha
        << isPalindrome("A man, a plan, a canal: Panama");
}

```

5. Count Characters Absent

Input: "banana"

Output: b=1, a=3, n=2

Main thing to learn: Scan the string and count each character.

Pattern / technique: Frequency counting

Complexity: Time $O(n)$ • Extra space $O(k)$, where k is the character set size

Note: Exact output order of `unordered_map` is unspecified.

Optimal C++

```

#include <iostream>
#include <string>

```

```
#include <unordered_map>
using namespace std;

unordered_map<char,int> countCharacters(const string& s) {
    unordered_map<char,int> freq;

    for (char c : s)
        freq[c]++;

    return freq;
}

int main() {
    auto freq = countCharacters("banana");

    for (auto [c, count] : freq)
        cout << c << "=" << count << " ";
}
```

6. Find Character Frequency **Absent**

Input: "programming", target = 'g'

Output: 2

Main thing to learn: Use a frequency table or count only the target while scanning.

Pattern / technique: Linear scan

Complexity: Time $O(n)$ • Extra space $O(1)$ for a fixed ASCII table

Optimal C++

```
#include <iostream>
#include <string>
using namespace std;

int frequency(const string& s, char target) {
    int count = 0;

    for (char c : s)
        if (c == target)
            count++;

    return count;
}

int main() {
    cout << frequency("programming", 'g');
}
```

7. First Non-Repeating Character **LeetCode 387**

Input: "leetcode"

Output: 0 (character 'l')

Main thing to learn: Count first, then scan again to find the first character with frequency 1.

Pattern / technique: Frequency + second scan

Complexity: Time $O(n)$ • Extra space $O(k)$

Note: Code assumes lowercase English letters, matching LeetCode 387 constraints.

Optimal C++

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int firstUniqChar(string s) {
    vector<int> freq(26, 0);

    for (char c : s)
        freq[c - 'a']++;

    for (int i = 0; i < s.size(); i++)
```

```

        if (freq[s[i] - 'a'] == 1)
            return i;

    return -1;
}

int main() {
    cout << firstUniqChar("leetcode");
}

```

8. Remove Duplicate Characters **Absent**

Input: "programming"

Output: "progamin"

Main thing to learn: Keep the first occurrence of each character and skip later duplicates.

Pattern / technique: Seen-set / frequency marking

Complexity: Time $O(n)$ • Extra space $O(k)$

Note: Assumes byte-sized characters; for Unicode, use an appropriate Unicode-aware approach.

Optimal C++

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

string removeDuplicates(string s) {
    vector<bool> seen(256, false);
    string ans;

    for (unsigned char c : s) {
        if (!seen[c]) {
            seen[c] = true;
            ans += char(c);
        }
    }

    return ans;
}

int main() {
    cout << removeDuplicates("programming");
}

```

9. Valid Anagram **LeetCode 242**

Input: s="anagram", t="nagaram"

Output: true

Main thing to learn: Two strings are anagrams when every character has the same frequency.

Pattern / technique: Frequency balance

Complexity: Time $O(n)$ • Extra space $O(1)$ for lowercase English letters

Note: Code assumes lowercase English letters.

Optimal C++

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;

    vector<int> freq(26, 0);

    for (char c : s) freq[c - 'a']++;
    for (char c : t) freq[c - 'a']--;

    for (int x : freq)

```

```

        if (x != 0)
            return false;

    return true;
}

int main() {
    cout << boolalpha << isAnagram("anagram", "nagaram");
}

```

10. Longest Common Prefix [LeetCode 14](#)

Input: ["flower","flow","flight"]

Output: "fl"

Main thing to learn: Compare characters column-by-column until a mismatch appears.

Pattern / technique: Vertical comparison

Complexity: Time $O(S)$ • Extra space $O(1)$, excluding output

Note: S is the total number of characters examined.

Optimal C++

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

string longestCommonPrefix(vector<string>& strs) {
    if (strs.empty()) return "";

    for (int i = 0; i < strs[0].size(); i++) {
        char c = strs[0][i];

        for (int j = 1; j < strs.size(); j++) {
            if (i >= strs[j].size() || strs[j][i] != c)
                return strs[0].substr(0, i);
        }
    }

    return strs[0];
}

int main() {
    vector<string> strs = {"flower", "flow", "flight"};
    cout << longestCommonPrefix(strs);
}

```

LEVEL 3 • HashMap / HashSet + Arrays

1. Two Sum [LeetCode 1](#)

Input: [2,7,11,15], target = 9

Output: [0,1]

Main thing to learn: For each value, look for target - value among previously seen values.

Pattern / technique: HashMap: value $\rightarrow$ index

Complexity: Time $O(n)$ average • Extra space $O(n)$

Optimal C++

```

#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int,int> mp;

    for (int i = 0; i < nums.size(); i++) {
        int need = target - nums[i];
    }
}

```

```

        if (mp.count(need))
            return {mp[need], i};

        mp[nums[i]] = i;
    }

    return {};
}

int main() {
    vector<int> nums = {2,7,11,15};
    auto ans = twoSum(nums, 9);
    cout << ans[0] << " " << ans[1];
}

```

2. Contains Duplicate LeetCode 217

Input: [1,2,3,1]

Output: true

Main thing to learn: Insert each value into a set; seeing it again means a duplicate exists.

Pattern / technique: HashSet

Complexity: Time $O(n)$ average • Extra space $O(n)$

Optimal C++

```

#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

bool containsDuplicate(vector<int>& nums) {
    unordered_set<int> seen;

    for (int x : nums) {
        if (seen.count(x))
            return true;
        seen.insert(x);
    }

    return false;
}

int main() {
    vector<int> nums = {1,2,3,1};
    cout << boolalpha << containsDuplicate(nums);
}

```

3. Majority Element LeetCode 169

Input: [2,2,1,1,1,2,2]

Output: 2

Main thing to learn: Boyer-Moore keeps a candidate and a vote count. A majority survives cancellation.

Pattern / technique: Boyer-Moore voting

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: LeetCode guarantees a majority element.

Optimal C++

```

#include <iostream>
#include <vector>
using namespace std;

int majorityElement(vector<int>& nums) {
    int candidate = 0, count = 0;

    for (int x : nums) {
        if (count == 0)
            candidate = x;
    }
}

```

```

        count += (x == candidate) ? 1 : -1;
    }

    return candidate;
}

int main() {
    vector<int> nums = {2,2,1,1,1,2,2};
    cout << majorityElement(nums);
}

```

4. Intersection of Two Arrays LeetCode 349

Input: [1,2,2,1] and [2,2]

Output: [2]

Main thing to learn: Put the first array into a set and collect values present in the second set.

Pattern / technique: HashSet

Complexity: Time $O(n+m)$ average • Extra space $O(n)$

Note: Intersection here is unique values, matching LC 349.

Optimal C++

```

#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

vector<int> intersection(vector<int>& a, vector<int>& b) {
    unordered_set<int> s(a.begin(), a.end());
    unordered_set<int> used;
    vector<int> ans;

    for (int x : b) {
        if (s.count(x) && !used.count(x)) {
            ans.push_back(x);
            used.insert(x);
        }
    }

    return ans;
}

int main() {
    vector<int> a = {1, 2, 2, 1};
    vector<int> b = {2, 2};

    for (int x : intersection(a, b))
        cout << x << " ";
}

```

5. Longest Consecutive Sequence LeetCode 128

Input: [100,4,200,1,3,2]

Output: 4

Main thing to learn: Only start counting from x when x-1 is absent. This prevents rescanning sequences.

Pattern / technique: HashSet + sequence starts

Complexity: Time $O(n)$ average • Extra space $O(n)$

Optimal C++

```

#include <iostream>
#include <vector>
#include <unordered_set>
#include <algorithm>
using namespace std;

int longestConsecutive(vector<int>& nums) {
    unordered_set<int> s(nums.begin(), nums.end());
    int best = 0;
}

```

```

    for (int x : s) {
        if (!s.count(x - 1)) {
            int y = x;
            while (s.count(y))
                y++;

            best = max(best, y - x);
        }
    }

    return best;
}

int main() {
    vector<int> nums = {100, 4, 200, 1, 3, 2};
    cout << longestConsecutive(nums);
}

```

6. Group Anagrams LeetCode 49

Input: ["eat","tea","tan","ate","nat","bat"]

Output: [["eat","tea","ate"],["tan","nat"],["bat"]]

Main thing to learn: Use a canonical key. Sorting each word is simple and groups equal signatures.

Pattern / technique: HashMap: sorted string → group

Complexity: Time $O(n \cdot k \log k)$ • Extra space $O(n \cdot k)$

Note: Output group order is not guaranteed by an `unordered_map`.

Optimal C++

```

#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> mp;

    for (string s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        mp[key].push_back(s);
    }

    vector<vector<string>> ans;

    for (auto& [key, group] : mp)
        ans.push_back(group);

    return ans;
}

int main() {
    vector<string> strs = {"eat", "tea", "tan", "ate", "nat", "bat"};
    auto ans = groupAnagrams(strs);

    for (auto& group : ans) {
        for (string& s : group) cout << s << " ";
        cout << "\n";
    }
}

```

7. Subarray Sum Equals K LeetCode 560

Input: [1,1,1], k = 2

Output: 2

Main thing to learn: Prefix sum p: a subarray ending here sums to k when an earlier prefix is p-k.

Pattern / technique: Prefix sum + frequency map

Complexity: Time $O(n)$ average • Extra space $O(n)$

Note: Works with negative numbers too.

Optimal C++

```
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

int subarraySum(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    freq[0] = 1;

    int prefix = 0, ans = 0;

    for (int x : nums) {
        prefix += x;

        if (freq.count(prefix - k))
            ans += freq[prefix - k];

        freq[prefix]++;
    }

    return ans;
}

int main() {
    vector<int> nums = {1,1,1};
    cout << subarraySum(nums, 2);
}
```

8. Find All Duplicates in an Array [LeetCode 442](#)

Input: [4,3,2,7,8,2,3,1]

Output: [2,3]

Main thing to learn: Use each value as an index and negate the value there. A second visit means duplicate.

Pattern / technique: Index marking

Complexity: Time $O(n)$ • Extra space $O(1)$, excluding output

Note: LeetCode constraints allow modifying nums.

Optimal C++

```
#include <iostream>
#include <vector>
#include <cstdlib>
using namespace std;

vector<int> findDuplicates(vector<int>& nums) {
    vector<int> ans;

    for (int x : nums) {
        int index = abs(x) - 1;

        if (nums[index] < 0)
            ans.push_back(abs(x));
        else
            nums[index] = -nums[index];
    }

    return ans;
}

int main() {
    vector<int> nums = {4,3,2,7,8,2,3,1};

    for (int x : findDuplicates(nums))
        cout << x << " ";
}
```

LEVEL 4 • Two Pointers

1. Valid Palindrome LeetCode 125

Input: "A man, a plan, a canal: Panama"

Output: true

Main thing to learn: Two pointers move inward while skipping non-alphanumeric characters.

Pattern / technique: Two pointers

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;

    while (l < r) {
        while (l < r && !isalnum(s[l])) l++;
        while (l < r && !isalnum(s[r])) r--;

        if (tolower(s[l]) != tolower(s[r]))
            return false;

        l++;
        r--;
    }

    return true;
}

int main() {
    cout << boolalpha
         << isPalindrome("A man, a plan, a canal: Panama");
}
```

2. Two Sum II LeetCode 167

Input: [2,7,11,15], target = 9

Output: [1,2]

Main thing to learn: The array is sorted. If the sum is too small, move left; if too large, move right.

Pattern / technique: Two pointers on sorted array

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: LeetCode uses 1-indexed answers.

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> twoSum(vector<int>& numbers, int target) {
    int l = 0, r = numbers.size() - 1;

    while (l < r) {
        int sum = numbers[l] + numbers[r];

        if (sum == target)
            return {l + 1, r + 1};
        if (sum < target)
            l++;
        else
            r--;
    }

    return {};
}

int main() {
    vector<int> numbers = {2,7,11,15};
    auto ans = twoSum(numbers, 9);
}
```

```
    cout << ans[0] << " " << ans[1];
}
```

3. Remove Duplicates from Sorted Array LeetCode 26

Input: [1,1,2,2,3]

Output: k = 3, unique prefix [1,2,3]

Main thing to learn: Slow pointer marks where the next unique value belongs.

Pattern / technique: Slow-fast pointers

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;

    int k = 1;

    for (int i = 1; i < nums.size(); i++)
        if (nums[i] != nums[i - 1])
            nums[k++] = nums[i];

    return k;
}

int main() {
    vector<int> nums = {1,1,2,2,3};
    int k = removeDuplicates(nums);
    cout << k << "\n";

    for (int i = 0; i < k; i++)
        cout << nums[i] << " ";
}
```

4. 3Sum LeetCode 15

Input: [-1,0,1,2,-1,-4]

Output: [[-1,-1,2],[-1,0,1]]

Main thing to learn: Sort first. Fix one value, then use two pointers for the remaining two values. Skip duplicates.

Pattern / technique: Sort + two pointers

Complexity: Time $O(n^2)$ • Extra space $O(1)$ excluding output (sorting implementation dependent)

Note: Duplicate skipping is essential.

Optimal C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> ans;

    for (int i = 0; i < nums.size(); i++) {
        if (i > 0 && nums[i] == nums[i - 1])
            continue;

        int l = i + 1, r = nums.size() - 1;

        while (l < r) {
            long long sum = 1LL * nums[i] + nums[l] + nums[r];

            if (sum == 0) {
                ans.push_back({nums[i], nums[l], nums[r]});
                l++;
            }
        }
    }
}
```

```

        r--;

        while (l < r && nums[l] == nums[l - 1]) l++;
        while (l < r && nums[r] == nums[r + 1]) r--;
    } else if (sum < 0) {
        l++;
    } else {
        r--;
    }
}

return ans;
}

int main() {
    vector<int> nums = {-1,0,1,2,-1,-4};

    for (auto& t : threeSum(nums)) {
        cout << "[" << t[0] << ", " << t[1] << ", " << t[2] << "]" ";
    }
}

```

5. Container With Most Water LeetCode 11

Input: [1,8,6,2,5,4,8,3,7]

Output: 49

Main thing to learn: Area is $\min(\text{height}[l], \text{height}[r]) * (r - l)$. Move the shorter side because moving the taller side cannot improve the limiting height.

Pattern / technique: Greedy two pointers

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int maxArea(vector<int>& height) {
    int l = 0, r = height.size() - 1;
    int best = 0;

    while (l < r) {
        int area = min(height[l], height[r]) * (r - l);
        best = max(best, area);

        if (height[l] < height[r])
            l++;
        else
            r--;
    }

    return best;
}

int main() {
    vector<int> height = {1,8,6,2,5,4,8,3,7};
    cout << maxArea(height);
}

```

6. Trapping Rain Water LeetCode 42

Input: [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Main thing to learn: Maintain leftMax and rightMax. Water at a side is bounded by the smaller known maximum.

Pattern / technique: Two pointers + running maxima

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int trap(vector<int>& h) {
    int l = 0, r = h.size() - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;

    while (l < r) {
        if (h[l] <= h[r]) {
            leftMax = max(leftMax, h[l]);
            water += leftMax - h[l];
            l++;
        } else {
            rightMax = max(rightMax, h[r]);
            water += rightMax - h[r];
            r--;
        }
    }

    return water;
}

int main() {
    vector<int> h = {0,1,0,2,1,0,1,3,2,1,2,1};
    cout << trap(h);
}

```

LEVEL 5 • Sliding Window

1. Maximum Sum Subarray of Size K **Absent**

Input: [2,1,5,1,3,2], k = 3

Output: 9

Main thing to learn: Keep a fixed-size window sum: add the new element and remove the element leaving the window.

Pattern / technique: Fixed-size sliding window

Complexity: Time $O(n)$ • Extra space $O(1)$

Note: No exact LeetCode problem with this exact 'maximum sum of size k' statement. LC 643 asks for maximum average instead.

Optimal C++

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

long long maxSumK(vector<int>& a, int k) {
    long long window = 0;

    for (int i = 0; i < k; i++)
        window += a[i];

    long long best = window;

    for (int r = k; r < a.size(); r++) {
        window += a[r];
        window -= a[r - k];
        best = max(best, window);
    }

    return best;
}

int main() {
    vector<int> a = {2,1,5,1,3,2};
    cout << maxSumK(a, 3);
}

```

2. Longest Substring Without Repeating Characters **LeetCode 3**

Input: "abcabcbb"

Output: 3

Main thing to learn: Expand right; if a character repeats, move left until the window is valid again.

Pattern / technique: Variable-size sliding window

Complexity: Time $O(n)$ • Extra space $O(k)$

Optimal C++

```
#include <iostream>
#include <string>
#include <unordered_set>
#include <algorithm>
using namespace std;

int lengthOfLongestSubstring(string s) {
    unordered_set<char> seen;
    int l = 0, best = 0;

    for (int r = 0; r < s.size(); r++) {
        while (seen.count(s[r])) {
            seen.erase(s[l]);
            l++;
        }

        seen.insert(s[r]);
        best = max(best, r - l + 1);
    }

    return best;
}

int main() {
    cout << lengthOfLongestSubstring("abcbcbbb");
}
```

3. Longest Repeating Character Replacement [LeetCode 424](#)

Input: s="AABABBA", k=1

Output: 4

Main thing to learn: Window is valid when window length - highest character frequency $\leq k$.

Pattern / technique: Variable sliding window + frequency

Complexity: Time $O(n)$ • Extra space $O(1)$ for uppercase English letters

Optimal C++

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

int characterReplacement(string s, int k) {
    vector<int> freq(26, 0);
    int l = 0, maxFreq = 0, best = 0;

    for (int r = 0; r < s.size(); r++) {
        maxFreq = max(maxFreq, ++freq[s[r] - 'A']);

        while ((r - l + 1) - maxFreq > k)
            freq[s[l++] - 'A']--;

        best = max(best, r - l + 1);
    }

    return best;
}

int main() {
    cout << characterReplacement("AABABBA", 1);
}
```

4. Permutation in String [LeetCode 567](#)

Input: s1="ab", s2="eidbaooo"

Output: true

Main thing to learn: Maintain a window of size |s1| and compare character frequency counts.

Pattern / technique: Fixed-size sliding window + frequency

Complexity: Time O(n) • Extra space O(1) for lowercase English letters

Optimal C++

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

bool checkInclusion(string s1, string s2) {
    if (s1.size() > s2.size()) return false;

    vector<int> need(26, 0), window(26, 0);

    for (char c : s1) need[c - 'a']++;

    int k = s1.size();

    for (int r = 0; r < s2.size(); r++) {
        window[s2[r] - 'a']++;

        if (r >= k)
            window[s2[r - k] - 'a']--;

        if (window == need)
            return true;
    }

    return false;
}

int main() {
    cout << boolalpha << checkInclusion("ab", "eidbaooo");
}
```

5. Fruit Into Baskets [LeetCode 904](#)

Input: [1,2,1]

Output: 3

Main thing to learn: Find the longest window containing at most two distinct values.

Pattern / technique: At-most-K distinct sliding window

Complexity: Time O(n) average • Extra space O(1) in the problem's value-domain interpretation

Optimal C++

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <algorithm>
using namespace std;

int totalFruit(vector<int>& fruits) {
    unordered_map<int, int> freq;
    int l = 0, best = 0;

    for (int r = 0; r < fruits.size(); r++) {
        freq[fruits[r]]++;

        while (freq.size() > 2) {
            if (--freq[fruits[l]] == 0)
                freq.erase(fruits[l]);
            l++;
        }

        best = max(best, r - l + 1);
    }

    return best;
}
```

```
int main() {
    vector<int> fruits = {1,2,1};
    cout << totalFruit(fruits);
}
```

6. Minimum Window Substring LeetCode 76

Input: s="ADOBECODEBANC", t="ABC"

Output: "BANC"

Main thing to learn: Expand until all required characters are present, then shrink from the left while staying valid.

Pattern / technique: Variable-size sliding window + counts

Complexity: Time $O(n)$ • Extra space $O(k)$

Optimal C++

```
#include <iostream>
#include <string>
#include <vector>
#include <climits>
using namespace std;

string minWindow(string s, string t) {
    vector<int> need(128, 0);
    for (char c : t) need[c]++;

    int missing = t.size();
    int l = 0, bestLen = INT_MAX, bestStart = 0;

    for (int r = 0; r < s.size(); r++) {
        if (need[s[r]] > 0)
            missing--;

        need[s[r]]--;

        while (missing == 0) {
            if (r - l + 1 < bestLen) {
                bestLen = r - l + 1;
                bestStart = l;
            }

            need[s[l]]++;

            if (need[s[l]] > 0)
                missing++;

            l++;
        }
    }

    return bestLen == INT_MAX ? "" : s.substr(bestStart, bestLen);
}

int main() {
    cout << minWindow("ADOBECODEBANC", "ABC");
}
```

LEVEL 6 • Prefix Sum + Kadane

1. Range Sum Query LeetCode 303

Input: nums=[-2,0,3,-5,2,-1], sumRange(0,2)

Output: 1

Main thing to learn: Build prefix sums so a range sum becomes $\text{prefix}[r+1] - \text{prefix}[l]$.

Pattern / technique: Prefix sum

Complexity: Build $O(n)$ • Query $O(1)$ • Extra space $O(n)$

Note: LC 303 supports repeated range queries efficiently.

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

class NumArray {
    vector<int> prefix;

public:
    NumArray(vector<int>& nums) {
        prefix.resize(nums.size() + 1, 0);

        for (int i = 0; i < nums.size(); i++)
            prefix[i + 1] = prefix[i] + nums[i];
    }

    int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
};

int main() {
    vector<int> nums = {-2, 0, 3, -5, 2, -1};
    NumArray obj(nums);

    cout << obj.sumRange(0, 2);
}
```

2. Find Pivot Index [LeetCode 724](#)

Input: [1,7,3,6,5,6]

Output: 3

Main thing to learn: Keep total sum. At index i, left sum is known and right sum = total - left - nums[i].

Pattern / technique: Prefix/total sum

Complexity: Time O(n) • Extra space O(1)

Optimal C++

```
#include <iostream>
#include <vector>
using namespace std;

int pivotIndex(vector<int>& nums) {
    int total = 0;

    for (int x : nums)
        total += x;

    int left = 0;

    for (int i = 0; i < nums.size(); i++) {
        int right = total - left - nums[i];

        if (left == right)
            return i;

        left += nums[i];
    }

    return -1;
}

int main() {
    vector<int> nums = {1, 7, 3, 6, 5, 6};
    cout << pivotIndex(nums);
}
```

3. Subarray Sum Equals K [LeetCode 560](#)

Input: [1,1,1], k=2

Output: 2

Main thing to learn: Use prefix-sum frequencies: currentPrefix - k tells how many earlier prefixes form a k-sum subarray.

Pattern / technique: Prefix sum + HashMap

Complexity: Time $O(n)$ average • Extra space $O(n)$

Optimal C++

```
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

int subarraySum(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    freq[0] = 1;

    int prefix = 0, ans = 0;

    for (int x : nums) {
        prefix += x;

        if (freq.count(prefix - k))
            ans += freq[prefix - k];

        freq[prefix]++;
    }

    return ans;
}

int main() {
    vector<int> nums = {1,1,1};
    cout << subarraySum(nums, 2);
}
```

4. Maximum Subarray [LeetCode 53](#)

Input: [-2,1,-3,4,-1,2,1,-5,4]

Output: 6

Main thing to learn: Kadane's algorithm decides whether to extend the current subarray or start fresh at the current element.

Pattern / technique: Kadane's algorithm

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int maxSubArray(vector<int>& nums) {
    int current = nums[0];
    int best = nums[0];

    for (int i = 1; i < nums.size(); i++) {
        current = max(nums[i], current + nums[i]);
        best = max(best, current);
    }

    return best;
}

int main() {
    vector<int> nums = {-2,1,-3,4,-1,2,1,-5,4};
    cout << maxSubArray(nums);
}
```

5. Maximum Sum Circular Subarray [LeetCode 918](#)

Input: [5,-3,5]

Output: 10

Main thing to learn: Answer is $\max(\text{normal Kadane}, \text{totalSum} - \text{minimumSubarray})$. Handle all-negative arrays separately.

Pattern / technique: Kadane for max + Kadane for min

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int maxSubarraySumCircular(vector<int>& nums) {
    int total = 0;
    int curMax = 0, maxSum = nums[0];
    int curMin = 0, minSum = nums[0];

    for (int x : nums) {
        curMax = max(x, curMax + x);
        maxSum = max(maxSum, curMax);

        curMin = min(x, curMin + x);
        minSum = min(minSum, curMin);

        total += x;
    }

    if (maxSum < 0)
        return maxSum;

    return max(maxSum, total - minSum);
}

int main() {
    vector<int> nums = {5, -3, 5};
    cout << maxSubarraySumCircular(nums);
}
```

LEVEL 7 • Important String Problems

1. Longest Palindromic Substring [LeetCode 5](#)

Input: "babad"

Output: "bab" ("aba" is also valid)

Main thing to learn: Expand around every possible center. Track the longest palindrome found.

Pattern / technique: Expand around center

Complexity: Time $O(n^2)$ • Extra space $O(1)$, excluding output

Note: Multiple outputs can be valid when tied.

Optimal C++

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

string longestPalindrome(string s) {
    if (s.empty()) return "";

    int start = 0, bestLen = 1;

    auto expand = [&](int l, int r) {
        while (l >= 0 && r < s.size() && s[l] == s[r]) {
            if (r - l + 1 > bestLen) {
                start = l;
                bestLen = r - l + 1;
            }
            l--;
            r++;
        }
    };

    for (int i = 0; i < s.size(); i++) {
        expand(i, i); // odd length
        expand(i, i + 1); // even length
    }
}
```

```

    return s.substr(start, bestLen);
}

int main() {
    cout << longestPalindrome("babad");
}

```

2. String Compression LeetCode 443

Input: [a,a,b,b,c,c,c]

Output: [a,2,b,2,c,3] (length 6)

Main thing to learn: Read a run, write its character, then write its count digits if count > 1.

Pattern / technique: Run-length encoding / two pointers

Complexity: Time $O(n)$ • Extra space $O(1)$

Optimal C++

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

int compress(vector<char>& chars) {
    int write = 0;
    int i = 0;

    while (i < chars.size()) {
        char c = chars[i];
        int j = i;

        while (j < chars.size() && chars[j] == c)
            j++;

        int count = j - i;
        chars[write++] = c;

        if (count > 1) {
            string num = to_string(count);

            for (char d : num)
                chars[write++] = d;
        }

        i = j;
    }

    return write;
}

int main() {
    vector<char> chars = {'a', 'a', 'b', 'b', 'c', 'c', 'c'};
    int len = compress(chars);

    for (int i = 0; i < len; i++)
        cout << chars[i] << " ";
    cout << "\nLength: " << len;
}

```

3. Longest Common Prefix LeetCode 14

Input: ["flower","flow","flight"]

Output: "fl"

Main thing to learn: Compare the same character position across all strings until one differs or ends.

Pattern / technique: Vertical string comparison

Complexity: Time $O(S)$ • Extra space $O(1)$, excluding output

Optimal C++

```

#include <iostream>
#include <vector>

```

```

#include <string>
using namespace std;

string longestCommonPrefix(vector<string>& strs) {
    if (strs.empty()) return "";

    for (int i = 0; i < strs[0].size(); i++) {
        for (int j = 1; j < strs.size(); j++) {
            if (i >= strs[j].size() || strs[j][i] != strs[0][i])
                return strs[0].substr(0, i);
        }
    }

    return strs[0];
}

int main() {
    vector<string> strs = {"flower", "flow", "flight"};
    cout << longestCommonPrefix(strs);
}

```

4. Isomorphic Strings [LeetCode 205](#)

Input: s="egg", t="add"

Output: true

Main thing to learn: Maintain a one-to-one mapping in both directions so two source characters cannot map to the same target.

Pattern / technique: Bidirectional mapping

Complexity: Time $O(n)$ • Extra space $O(k)$

Optimal C++

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

bool isIsomorphic(string s, string t) {
    vector<int> st(256, -1), ts(256, -1);

    for (int i = 0; i < s.size(); i++) {
        unsigned char a = s[i], b = t[i];

        if (st[a] != -1 && st[a] != b)
            return false;

        if (ts[b] != -1 && ts[b] != a)
            return false;

        st[a] = b;
        ts[b] = a;
    }

    return true;
}

int main() {
    cout << boolalpha << isIsomorphic("egg", "add");
}

```

5. Find All Anagrams in a String [LeetCode 438](#)

Input: s="cbaebabacd", p="abc"

Output: [0,6]

Main thing to learn: Use a fixed-size window of |p| and maintain character frequency balance.

Pattern / technique: Fixed-size sliding window + frequency

Complexity: Time $O(n)$ • Extra space $O(1)$ for lowercase English letters

Optimal C++

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

vector<int> findAnagrams(string s, string p) {
    vector<int> need(26, 0), window(26, 0);
    vector<int> ans;

    if (p.size() > s.size()) return ans;

    for (char c : p)
        need[c - 'a']++;

    int k = p.size();

    for (int r = 0; r < s.size(); r++) {
        window[s[r] - 'a']++;

        if (r >= k)
            window[s[r - k] - 'a']--;

        if (window == need)
            ans.push_back(r - k + 1);
    }

    return ans;
}

int main() {
    for (int i : findAnagrams("cbaebabacd", "abc"))
        cout << i << " ";
}

```

FINAL PATTERN MAP

Linear scan: maximum, minimum, sorted check, target/index, occurrence.

Two pointers: reverse array, palindrome, Two Sum II, 3Sum, container, trapping rain water.

HashMap / HashSet: Two Sum, duplicate checks, majority, intersection, consecutive sequence, anagrams, prefix-sum counting.

Index marking / placement: duplicate and missing-number families.

Sliding window: fixed-size and variable-size windows with a validity condition.

Prefix sum: range queries, pivot index, and subarray-sum counting.

Kadane: maximum subarray and circular maximum subarray.

String center expansion: longest palindromic substring.

Run-length encoding: string compression.